

Experiment Tracking System

Goal Apply OOP design concepts in a new problem — not by copying the class examples used in the session.

SCENARIO An AI research team runs many experiments and needs a small system to organize, compare, and report their results.

1. Experiment

- Create an Experiment class with: name, researcher, and score.
- score must always remain between 0 and 1; invalid values must be rejected.
- All experiments belong to "ITI AI Lab". Store this once for the class, not separately in every object.
- Provide an alternative way to create an experiment from a dictionary such as:

```
experiment_data = {
    "name": "Sentiment Analysis",
    "researcher": "Ahmed",
    "score": 0.87
}
```

Decision Choose the appropriate method type for creating an object from the dictionary.

2. Different Experiment Types

Create two specialized experiment types that reuse common Experiment behavior:

Type	Extra Data	report() Behavior
ClassificationExperiment	number_of_classes	Classification-specific report
RetrievalExperiment	top_k	Retrieval-specific report

Requirement Reuse the parent initialization and override report() so the same method name behaves differently for each experiment type.

3. Experiment Dashboard

- Create ExperimentDashboard to store experiment objects.
- It should provide: add_experiment(experiment), show_reports(), and best_experiment().
- show_reports() must work with both experiment types without checking their exact class using if/instance.
- best_experiment() returns the experiment with the highest score.

Example outcome Sentiment Analysis (0.87), Arabic RAG Search (0.91), Image Classification (0.83) → Best: Arabic RAG Search.

★ **BONUS** The dashboard should reject two experiments with the same name. The dashboard itself must be responsible for maintaining this rule.

لا تنسونا من صالح دعائكم